IIT JODHPUR · SEM 2 · APPLIED AI

Numerical Optimisation

4-Week Practical Session — Complete Revision Guide

This document covers all practical session content from Weeks 1–4 of the Numerical Optimisation course. It is structured for use as a NotebookLM source: each week is self-contained with concepts, code patterns, formulae, and exam Q&A.

Week	Topic	Key Ideas
Week 1	Data → Visual Intuition	NumPy arrays, scatter/line plots, manual y=mx+c fitting, polyfit
Week 2	Gradient Descent (1D)	Update rule, learning rate, stepping down 1D functions
Week 3	GD on Real Data	Convex vs non-convex, feature normalisation, MSE loss, linear regression
Week 4	LR Problems + Backtracking	Divergence, slow convergence, backtracking line search

THE CONNECTING THREAD

Week 1 established the *problem*: you manually guessed m and c to fit a line to data. Numerical Optimisation is about replacing that guessing with systematic algorithms.

Week 2 introduced the solution: gradient descent. The slope of a function tells you which direction to move; take a step opposite to it; repeat until you reach the minimum.

Week 3 applied GD to a real regression problem where m and c are the parameters being optimised, MSE is the loss, and feature normalisation is needed for stable convergence.

Week 4 exposed what breaks when the learning rate is wrong, and introduced backtracking line search as an adaptive fix — the conceptual ancestor of Adam, RMSProp, etc.

WEEK 01

Data → Visual Intuition

numpy arrays · matplotlib · manual line fitting · y = mx + c

1.1 NumPy Arrays

NumPy is the foundation of scientific Python. Arrays have a shape property that tells you their dimensions. Key creation functions:

- **np.array([...])** — create from a Python list
- **np.linspace(a, b, n)** — n evenly spaced values between a and b (inclusive). Use for smooth curves.
- **np.arange(a, b)** — integers from a to b-1, like Python range(). Use for index sequences.
- **arr.shape** — returns (rows, cols) for 2D, (n,) for 1D

Example from session — marks of multiple students:

```
marks_matrix = np.array([[85, 78, 95],
                         [60, 72, 88],
                         [91, 84, 89]])
print(marks_matrix.shape)  # (3, 3)
```

1.2 Plotting: plt.plot() vs plt.scatter()

The two most important Matplotlib functions for this course:

- **plt.scatter(x, y)** — shows individual data points. Use for raw observations where points are independent (e.g. one student's marks, one house's price).
- **plt.plot(x, y)** — connects points with a line. Use for model lines, function curves, or data over time.
- **Always label:** plt.xlabel(), plt.ylabel(), plt.title(), plt.legend() — noted in your handwritten notes too.

1.3 Manual Line Fitting (hit and trial)

The core exercise of Week 1: given scatter data, you guessed values of m and c in y = m*x + c until the line visually fit the points. For the house price dataset:

```
price = m x size + c
m = 0.1089,   c = -19.97   (your best guess)
```

This manual process IS the problem that the rest of the course solves automatically. np.polyfit() gives the mathematically optimal values — Weeks 3 and 4 implement this from scratch.

```
# Scatter + manual line
sizes  = np.array([650, 800, 1200, 1500, 2000])
prices = np.array([50, 70, 110, 140, 200])
plt.scatter(sizes, prices, label='data')
m, c = 0.1089, -19.97
line = m * sizes + c
plt.plot(sizes, line, color='green', label='Our Hint')
plt.legend()
plt.show()
```

EXAM Q&A; — Week 1

Q: What is the difference between np.linspace and np.arange?

A: np.linspace(a, b, n) gives exactly n evenly-spaced values between a and b (inclusive). np.arange(a, b) gives integers from a up to b-1 with step 1. Use linspace for smooth curves and arange for integer index sequences.

Q: When would you use plt.scatter vs plt.plot?

A: Use plt.scatter for raw data points where each observation is independent. Use plt.plot for model lines or any sequence where points should be connected. In Week 1 we combined both: scatter for data, plot for the guessed line.

Q: What problem does Numerical Optimisation actually solve?

A: It automates the search for optimal parameters (like m and c) that minimise a cost function. Week 1 showed this done manually by trial and error. NO gives systematic algorithms — like gradient descent — to find these values without guessing.

WEEK 02

Gradient Descent (1D)

the core algorithm · learning rate · stepping down a curve

2.1 The Core Idea

If you are standing on a hill and want to reach the lowest point: look at which way the ground slopes under your feet, then take a step in the OPPOSITE direction. Repeat. That is gradient descent.

The algorithm uses the derivative (gradient) of a function to decide which direction is 'downhill', then takes a step in that direction. It does not need to see the whole function — only the local slope at the current point.

2.2 The Update Rule — memorise this

```
x_new = x_old - learning_rate x gradient(x_old)
```

- **gradient(x)** = the derivative of your function at x — tells you the slope direction
- **Minus sign** = we go OPPOSITE to the slope (downhill)
- **learning_rate (lr)** = controls how big a step to take each iteration

2.3 Example 1: f(x) = x squared, minimum at x = 0

The derivative (gradient) of x^2 is 2x. Starting at x = -7 with lr = 0.05:

```
def f(x):    return x**2
def grad(x): return 2*x

x = -7
for i in range(20):
    slope = grad(x)
    x = x - 0.05 * slope
# x converges to 0 (the minimum)
```

2.4 Example 2: f(x) = (x-3) squared + 1, minimum at x = 3

Gradient = 2(x-3). Starting at x = 0.5, lr = 0.1, 20 iterations:

```
def f(x):     return (x-3)**2 + 1
def df_dx(x):  return 2*(x-3)

x = 0.5
for i in range(20):
    x = x - 0.1 * df_dx(x)
# x converges to 3
```

```
When gradient = 0  -->  x = x - lr*0 = x  -->  no movement
This is the stopping condition (stationary point)
```

2.5 Learning Rate Effects

• **lr too small (e.g. 0.001)** — tiny steps, converges correctly but extremely slowly. Needs many more iterations.

• **lr too large (e.g. 0.5+)** — overshoots minimum, bounces to the other side, may diverge (cost increases). This is the problem Week 4 solves.

• **lr just right** — fast, smooth descent to the minimum.

• **slope = 0** — algorithm stops moving. For convex functions this is always the global minimum. For non-convex functions it could be a local min, local max, or saddle point.

EXAM Q&A; — Week 2

Q: Write the gradient descent update rule and explain each term.

A: x = x - lr * gradient(x). Here x is the current parameter, lr is the learning rate (controls step size), gradient(x) is the derivative at x giving direction of steepest ascent, and the minus sign moves us opposite to it (downhill).

Q: For f(x) = (x-5) squared + 3, what is the gradient? Where is the minimum?

A: Gradient = 2(x-5). Minimum is at x = 5, where f(5) = 3. The gradient is zero there so GD stops moving.

Q: What effect does the learning rate have on GD convergence?

A: Too small → convergence is painfully slow, needs many iterations. Too large → overshoots minimum, algorithm may diverge with cost increasing. Just right → fast, stable convergence. This motivates adaptive methods like backtracking (Week 4).

WEEK 03

GD on Real Data

convexity · feature normalisation · MSE loss · linear regression

---

3.1 Convex vs Non-Convex Functions

This is critical for understanding when gradient descent is trustworthy:

- **Convex function** (e.g. x^2, |x|): bowl shape. Any local minimum is also the global minimum. GD is guaranteed to find the best answer regardless of starting point.
- **Non-convex function** (e.g. sin(x), x^3 - 3x): has multiple valleys. GD can get stuck in a local minimum that is not globally optimal. Starting point matters.
- **Why it matters for AI**: MSE loss for linear regression is convex — GD always finds the true best fit. Neural network loss is non-convex — GD finds a good solution but not necessarily the best one.

```
x = np.linspace(-10, 10, 200)
y_convex    = x**2       # bowl
y_nonconvex = np.sin(x)*10 # multiple valleys

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(x, y_convex,    color='blue')   # Week 3 also showed |x| and x^3-3x
ax[1].plot(x, y_nonconvex, color='red')
plt.show()
```

3.2 Feature Normalisation

House sizes are in the 1200-2700 range; prices are 45-130. This scale mismatch causes gradient descent to diverge or require an extremely small learning rate. The fix:

```
sizes_norm = (sizes - sizes.mean()) / sizes.std()
```

This is called standardisation (z-score normalisation). It centres the data around 0 with a standard deviation of 1. After normalisation, GD converges stably with a normal learning rate like 0.01.

3.3 Mean Squared Error (MSE) — the loss function

MSE measures how wrong our model is. It is what gradient descent minimises:

```
MSE = (1/n) x sum( (y_predicted - y_actual)^2 )
```

- **Why squaring?** (1) removes negatives so positive and negative errors don't cancel, (2) penalises large errors more severely (error of 10 contributes 100, not just 10).
- **Why MSE is ideal here:** it is convex and differentiable, so GD is guaranteed to find the globally optimal m and c.

3.4 Gradients of MSE with respect to m and c

These come from differentiating MSE with respect to each parameter:

```
dm = -2 * mean( X * (y - y_pred) )
dc = -2 * mean( y - y_pred )
```

```
m = m - lr * dm
c = c - lr * dc
```

3.5 Full Gradient Descent Loop for Linear Regression

```
m, c = 0, 0
lr = 0.1
num_epochs = 50

for epoch in range(num_epochs):
    y_pred = m * X + c                        # predict
    mse    = np.mean((y - y_pred)**2)          # compute loss
    dm     = -2 * np.mean(X * (y - y_pred))   # gradient for m
    dc     = -2 * np.mean(y - y_pred)          # gradient for c
    m = m - lr * dm                            # update m
    c = c - lr * dc                            # update c
```

The 5-step loop pattern: INIT → PREDICT → LOSS → GRADIENTS → UPDATE. Each pass through the data is called an epoch. MSE should decrease each epoch.

EXAM Q&A; — Week 3

Q: Why do we normalise features before gradient descent?

A: When features have very different scales, the cost function forms elongated ellipses in parameter space and GD takes many tiny zigzag steps. Normalising brings all features to the same scale (mean=0, std=1), making the cost surface more circular. GD converges faster and is stable with a larger learning rate.

Q: Why is MSE a good loss function for regression? Why square the errors?

A: (1) Squaring removes negatives — positive and negative errors don't cancel. (2) Squaring penalises large errors more — error of 10 contributes 100. (3) MSE is convex and differentiable — GD is guaranteed to find the global minimum. (4) The mean makes it independent of dataset size.

Q: What are epochs and why track MSE across them?

A: An epoch is one full pass through the training data — one complete round of predict, compute loss, update. Tracking MSE across epochs verifies the model is learning: MSE should decrease and eventually plateau. MSE increasing means lr is too large; MSE decreasing very slowly means lr is too small.

WEEK 04

LR Problems + Backtracking Line Search

divergence · slow convergence · adaptive step sizing

4.1 Problem A: Learning Rate Too Large (Divergence)

With lr = 0.75 on the house price dataset:

- The update step overshoots the minimum — jumps past the valley bottom to the other side.
- Cost at the new point is HIGHER than before. Next step overshoots again, further away.
- Result: cost grows exponentially, parameters become infinite / NaN. Algorithm fails.

```
Cost: 10000 --> 1000000 --> 1e12 ...  DIVERGED
```

4.2 Problem B: Learning Rate Too Small (Painfully Slow)

With lr = 0.001 on the same dataset:

- Takes tiny steps in the right direction. Converges correctly but extremely slowly.
- For simple problems this is merely annoying. In deep learning with millions of parameters, this wastes enormous amounts of GPU compute.

```
Cost: 10000 --> 9998 --> 9996 ...  (50 iterations, barely moved)
```

4.3 The Fix: Backtracking Line Search

Instead of using a fixed learning rate, start large and shrink automatically until the cost actually decreases. The algorithm:

- Step 1: Compute current cost at (m, c).
- Step 2: Try a step with a large initial learning rate (e.g. 1.0).
- Step 3: Compute new cost after the step.
- Step 4: If new cost < old cost → ACCEPT the step. Move to next iteration.
- Step 5: If new cost >= old cost → shrink: lr = lr * shrink_factor (e.g. 0.5). Go to Step 2.

```
while cost_new >= cost_old:
    lr = lr * shrink_factor
    # try step again with smaller lr
```

4.4 Backtracking Implementation Pattern

```
def gradient_descent_backtracking(X, y, initial_lr=1.0, iterations=150, shrink_facto
r=0.5):
    m, c = 0, 0
    for _ in range(iterations):
        current_cost = compute_cost(X, y, m, c)
        # compute gradients
        error = m*X + c - y
        dm = (2/len(X)) * np.dot(error, X)
```

```
        dc = (2/len(X)) * np.sum(error)
        # backtracking
        lr = initial_lr
        while compute_cost(X, y, m - lr*dm, c - lr*dc) >= current_cost:
            lr *= shrink_factor
        m = m - lr * dm
        c = c - lr * dc
    return m, c
```

4.5 Key Guarantee and Broader Significance

Backtracking guarantees that cost NEVER increases between iterations — divergence is impossible by construction. It uses large steps when the cost surface is smooth (fast progress early on) and automatically shrinks near the minimum (precision).

This is the conceptual ancestor of modern adaptive optimisers. Adam and RMSProp extend this idea further — they maintain a running estimate of gradient magnitudes per parameter and scale each parameter's learning rate individually. Week 4 is why those methods exist.

EXAM Q&A; — Week 4

Q: What happens with lr = 0.75 on the house price GD problem? Why?

A: The update step overshoots the minimum — it jumps past the valley bottom to the other side where cost is higher. The next step overshoots further. This causes divergence: cost grows exponentially and parameters blow up. It happens because the step size exceeds what the curvature of the cost surface can accommodate.

Q: Describe the backtracking line search algorithm in plain language.

A: (1) Compute current cost. (2) Try a step with a large initial learning rate. (3) If cost decreased, accept the step. (4) If cost increased or stayed same, shrink lr by shrink_factor and try again. (5) Repeat shrinking until cost decreases. This guarantees cost never increases, and uses large steps when safe.

Q: How does backtracking line search relate to modern optimisers like Adam?

A: Both solve the same problem: choosing a good step size adaptively rather than using a fixed lr. Backtracking adjusts lr per iteration based on whether cost improved. Adam and RMSProp go further — they maintain per-parameter lr estimates based on historical gradient magnitudes. Week 4 is the foundational motivation for these methods.

RECOMMENDED RESOURCES

Videos, Courses & References

curated for Applied AI students at IIT Jodhpur

Resource	Wk	Why it's relevant	Where / URL
3Blue1Brown Essence of Calculus (Ep 6–8: Derivatives)	2	Best visual intuition for why derivatives equal slope — the entire foundation of gradient descent.	youtube.com/@3blue1brown
3Blue1Brown Neural Networks series (Ep 2: Gradient Descent)	2–3	The clearest visual explanation of the GD update rule. Directly mirrors your Week 2–3 practical code.	youtube.com/@3blue1brown
StatQuest Gradient Descent, Step by Step	2	Slower-paced walkthrough of 1D gradient descent with explicit numbers. Great complement to the 3B1B visuals.	youtube.com/@statquest
Andrej Karpathy micrograd (YouTube)	3–4	Builds backpropagation from scratch in Python. Shows exactly how gradients flow — deeper context for dm and dc.	youtube.com/@AndrejKarpathy
Khan Academy Multivariable Calculus (Partial Derivatives)	3	The maths behind the dm and dc formulas from Week 3. Partial derivatives explained from first principles.	khanacademy.org
Sebastian Lague Gradient Descent (visual simulation)	3–4	Animated 3D cost surface with GD stepping across it. Makes divergence and convergence behaviour instantly obvious.	youtube.com/@SebastianLague
Coursera ML Specialisation — Andrew Ng (Week 1–2)	3	The most famous ML course. Linear regression + gradient descent coverage is almost identical to your Week 3 session.	coursera.org (free to audit)
NumPy Official Quickstart	1	Solidify Week 1 array operations: shapes, linspace, arange, broadcasting. 15-minute read.	numpy.org/doc/stable/ user/quickstart.html
Matplotlib Cheat Sheet (official)	1	All plot customisation options — colours, markers, linestyles, subplots. Useful for every practical session.	matplotlib.org/cheatsheets

QUICK FORMULA REFERENCE

Formula	Meaning	Week
`y = m*x + c`	Linear model (line equation)	1
`x = x - lr * grad(x)`	GD update rule (1D)	2
`grad of x^2 = 2x`	Derivative of x squared	2
`grad of (x-a)^2+b = 2(x-a)`	Derivative of shifted parabola	2

`MSE = (1/n)*sum((y_pred - y)^2)`	Mean Squared Error loss	3
`dm = -2*mean(X*(y - y_pred))`	MSE gradient w.r.t. m	3
`dc = -2*mean(y - y_pred)`	MSE gradient w.r.t. c	3
`sizes_norm = (sizes-mean)/std`	Feature normalisation (z-score)	3
`if cost_new >= cost_old: lr *= 0.5`	Backtracking shrink condition	4